

PROJECT REPORT

GitHub System

Users • Organizations • Repositories • Contributors

Submitted by:

Adarasha Gaihre

**BSc.CSIT 5th sem
Batch:2080**

Development Methodology: Waterfall Model

1: Introduction

1.1 Introduction

1.2 Problem Statement

1.2.1 Lack of Centralized Collaboration _____4

1.2.2 Absence of Version History and Traceability _____4

1.2.3 Barriers to Open-Source Contribution _____4

1.3 Objectives

1.4 Scope and Limitations

1.4.1 Scope _____5

1.4.2 Limitations _____5

1.5 Development Methodology: Waterfall Model

Chapter 2: Background Study and Literature Review

2.1 Background Study

2.1.1 Core Entity Overview _____6

2.2 Literature Review

Chapter 3: System Analysis

3.1 Requirement Analysis

3.1.1 Functional Requirements _____7

3.1.2 Non-Functional Requirements _____7

3.2 Feasibility Analysis

3.2.1 Economic Feasibility _____7

3.2.2 Schedule Feasibility _____7

3.2.3 Technical Feasibility _____8

3.2.4 Operational Feasibility _____8

3.3 Context Diagram

3.4 Data Flow Diagram

Level-0 DFD (System Overview) _____9

Level-1 DFD — Repository Management (Process 2.0) _____	10
Level-1 DFD — Contribution Workflow (Process 4.0) _____	10
Level-2 DFD — Pull Request Processing (Sub-process 4.3-4.4) ____	11
3.5 Entity Relationship Diagram (ERD)	
3.5.1 Physical Database Table Design _____	13
3.6 Sequence Diagram	
Chapter 4: Implementation and Maintenance	
4.1 Pseudo Code	
4.2 Test Cases	
Chapter 5: Conclusion	
5.1 Conclusion	
5.2 Lessons Learned	
5.3 Future Recommendations	
References	

1: Introduction

1.1 Introduction

GitHub is the world's largest web-based platform for version control and collaborative software development, built on top of Git. Launched in 2008 and acquired by Microsoft in 2018, GitHub hosts millions of repositories and serves as the backbone of modern open-source development worldwide.

This report analyzes a focused subset of the GitHub system: Users, Organizations, Repositories, and Contributors. These four entities form the core of GitHub's collaborative model. The system manages how individual users create and own repositories, how organizations group members under shared projects, and how contributors interact with repositories through code commits, pull requests, and issues.

1.2 Problem Statement

Before platforms like GitHub, software teams faced significant challenges in collaborative development:

1.2.1 Lack of Centralized Collaboration

Traditional development relied on emailing code files or using shared network drives. There was no reliable mechanism for multiple developers to work on the same codebase simultaneously without overwriting changes. This led to conflicts, lost work, and significant coordination overhead, especially for distributed teams working across different time zones.

1.2.2 Absence of Version History and Traceability

Without a version control platform, tracking who made a change, when, and why was nearly impossible. Teams maintained manual change logs or resorted to naming files with version suffixes (e.g., `project_final_v2_FINAL.zip`). This lack of traceability made debugging and rollback extremely difficult and costly.

1.2.3 Barriers to Open-Source Contribution

Potential contributors to open-source projects had no standardized workflow for proposing changes. Project maintainers had no efficient way to review, discuss, and merge external contributions. This created a fragmented ecosystem with high friction for both contributors and maintainers, slowing the pace of collaborative software innovation globally.

1.3 Objectives

The primary objectives of this system are:

- Enable users to register, authenticate, and manage repositories and access tokens.
- Allow organizations to group users under teams with defined roles and granular permissions.
- Provide repository management including code storage, branching, commit history, and issue tracking.
- Facilitate contributor workflows through forking, pull requests, code review, and merge operations.
- Ensure data integrity, role-based access control, and real-time synchronization across all operations.

1.4 Scope and Limitations

1.4.1 Scope

- User Management: Registration, authentication, profile management, SSH/HTTPS access configuration.
- Organization Management: Creating organizations, managing teams, assigning member roles (Owner, Member, Billing Manager).
- Repository Management: Creating public/private repositories, branching, commit history, file management.
- Contributor Workflow: Forking, pull requests, code review cycles, issue tracking, and merge operations.

1.4.2 Limitations

- This analysis does not cover GitHub Actions (CI/CD), GitHub Packages, or billing/subscription management.
- Advanced security features such as Dependabot, secret scanning, and code scanning are outside scope.
- Offline Git operations at the local machine level are not modeled in the system diagrams.

1.5 Development Methodology: Waterfall Model

The Waterfall methodology is selected for this analysis because the scope is limited to four well-defined entities with stable, clearly documentable requirements. The sequential phases are: Requirements Gathering and Analysis, System Design (DFDs, ERD, Sequence Diagrams), Implementation (Pseudo Code, Database Schema), Testing (Test Cases), and Maintenance Planning. This structured approach minimizes ambiguity and ensures thorough documentation at each phase before proceeding to the next.

Chapter 2: Background Study and Literature Review

This chapter reviews the background and relevant literature for the GitHub core system analysis, focusing on version control evolution, collaborative development workflows, and research influencing modern platform design.

2.1 Background Study

Version control systems have evolved from manual file management to distributed, cloud-based collaboration platforms. Early VCS tools like CVS and Subversion introduced centralized version tracking but created bottlenecks around the central server. The release of Git by Linus Torvalds in 2005 introduced a distributed model where every developer holds a complete repository copy. GitHub built a social collaboration layer on top of Git, introducing forks, pull requests, and issues as the standard open-source workflow.

2.1.1 Core Entity Overview

- **Users:** Individual developer accounts that own repositories and participate in projects.
- **Organizations:** Shared accounts grouping users under teams with defined permissions.
- **Repositories:** Core storage units holding code, history, issues, and project documentation.
- **Contributors:** Users who participate in a repository via commits, pull requests, and issue interactions.

2.2 Literature Review

Kalliamvakou et al. conducted a comprehensive study of GitHub as a research platform, finding that pull request workflows and contributor activity patterns are the most significant indicators of project health and community engagement. Their work highlights the importance of transparent contribution tracking and role clarity in collaborative platforms.

Tsay et al. examined factors influencing whether pull requests are accepted on GitHub, finding that established contributor-maintainer relationships and clear code quality signals significantly affect merge outcomes. This reinforces the need for a well-modeled PR status lifecycle in any GitHub system design.

Bird et al. found that large organizations using enterprise Git platforms benefit most from hierarchical team structures and granular permission controls, validating the importance of the Organization-Membership-Role model presented in this report's ERD.

Collectively, these studies support a system design that centers on clear entity relationships, transparent contributor workflows, and scalable permission models, all of which are reflected in the DFDs and ERD in Chapter 3.

Chapter 3: System Analysis

3.1 Requirement Analysis

3.1.1 Functional Requirements

- The system shall allow users to register, authenticate via email/password or OAuth, and manage their profiles including bio, avatar, and linked SSH keys.
- The system shall enable users to create, configure, and delete repositories with visibility settings (public or private) and default branch configuration.
- The system shall allow organizations to be created, with owners able to add or remove members, create teams, and assign repository access roles.
- The system shall track contributors per repository, recording commit counts, pull request history, and issue assignments.
- The system shall support forking — creating a personal copy of another user's or organization's repository.
- The system shall manage pull requests with a full lifecycle: open, review, approve/request-changes, merge, and close.
- The system shall send notifications to users for pull request reviews, issue assignments, and org membership invitations.

3.1.2 Non-Functional Requirements

- Performance: The system must handle millions of concurrent repository operations with sub-second response times for common actions.
- Security: The system must enforce role-based access control at organization and repository levels, with all data encrypted via HTTPS/SSH.
- Usability: Contribution workflows (fork, branch, pull request) must follow a clear, consistent pattern accessible to both novice and enterprise users.
- Scalability: Architecture must support horizontal scaling to accommodate growing users, organizations, and repositories without degradation.
- Availability: The system must maintain 99.9% uptime with redundant infrastructure for authentication and repository access services.

3.2 Feasibility Analysis

3.2.1 Economic Feasibility

The core GitHub model is economically viable through a freemium tier structure. Public repositories are free, while private repositories and advanced organizational features are offered through paid plans. Automation of repository management, issue tracking, and contribution workflows significantly reduces manual overhead compared to ad-hoc solutions, providing measurable cost savings for development teams.

3.2.2 Schedule Feasibility

The four core entities represent a well-scoped system with clearly defined relationships. A phased approach — starting with user authentication and repository management, followed by organization and contributor workflows — allows for incremental delivery within a realistic timeline, with each phase building on a tested prior layer.

3.2.3 Technical Feasibility

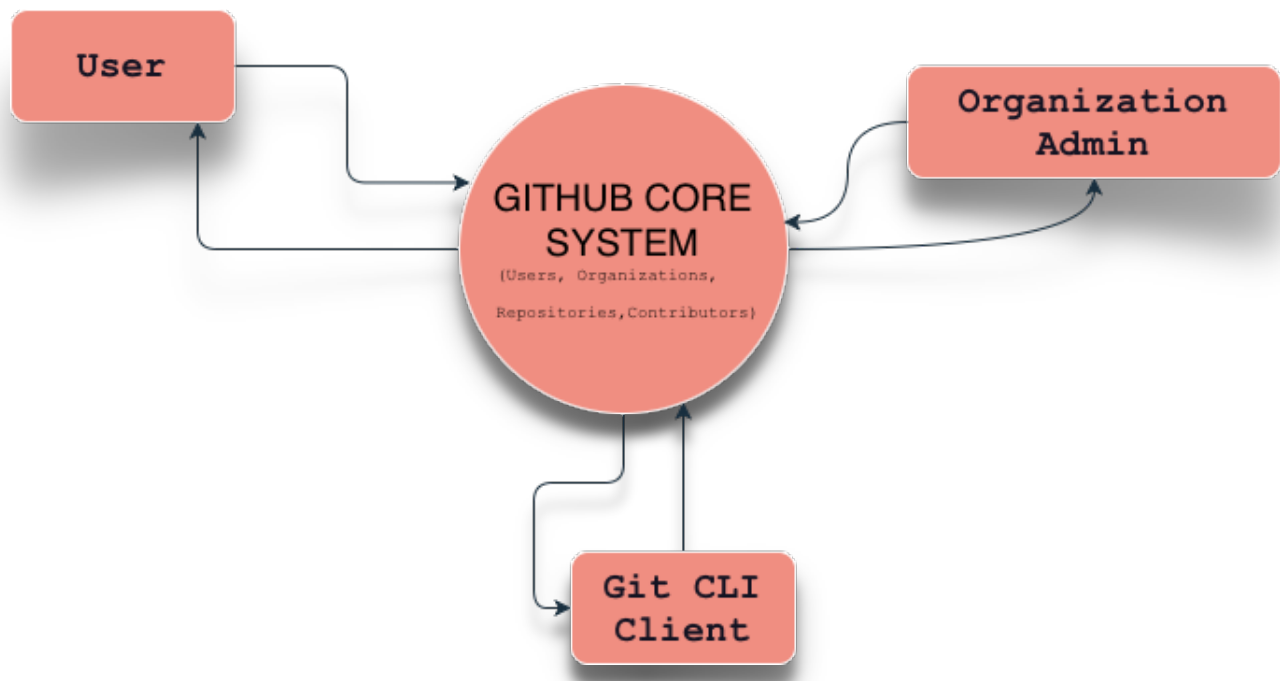
The system uses proven, widely available technologies: Git for distributed version control, PostgreSQL for relational metadata storage, REST and GraphQL APIs for client communication, and OAuth 2.0 for third-party authentication. All required technologies are mature, well-documented, and widely supported by existing developer tooling.

3.2.4 Operational Feasibility

GitHub's model has demonstrated strong operational feasibility at scale, serving over 100 million developers globally. The fork-commit-pull request workflows have become industry-standard practices, indicating high user acceptance and minimal training overhead. Organizations have successfully integrated GitHub into existing DevOps pipelines with minimal disruption.

3.3 Context Diagram

The context diagram defines the system boundary and interactions with external actors. Primary external entities are: User (individual developer), Organization Admin, and Git CLI Client.



Data flows:

User --> System	: Register, create repo, push commits, submit PR
System --> User	: Repo content, PR status, notifications
Org Admin --> System	: Team creation, member management, permissions
Git CLI --> System	: Push/pull via HTTPS or SSH

Fig: Context Diagram for GitHub Core System

3.4 Data Flow Diagram

The following DFDs represent how data flows through the GitHub core system at three levels of detail. Special attention is given to the Repository Management and Contribution Workflow processes.

Level-0 DFD (System Overview)

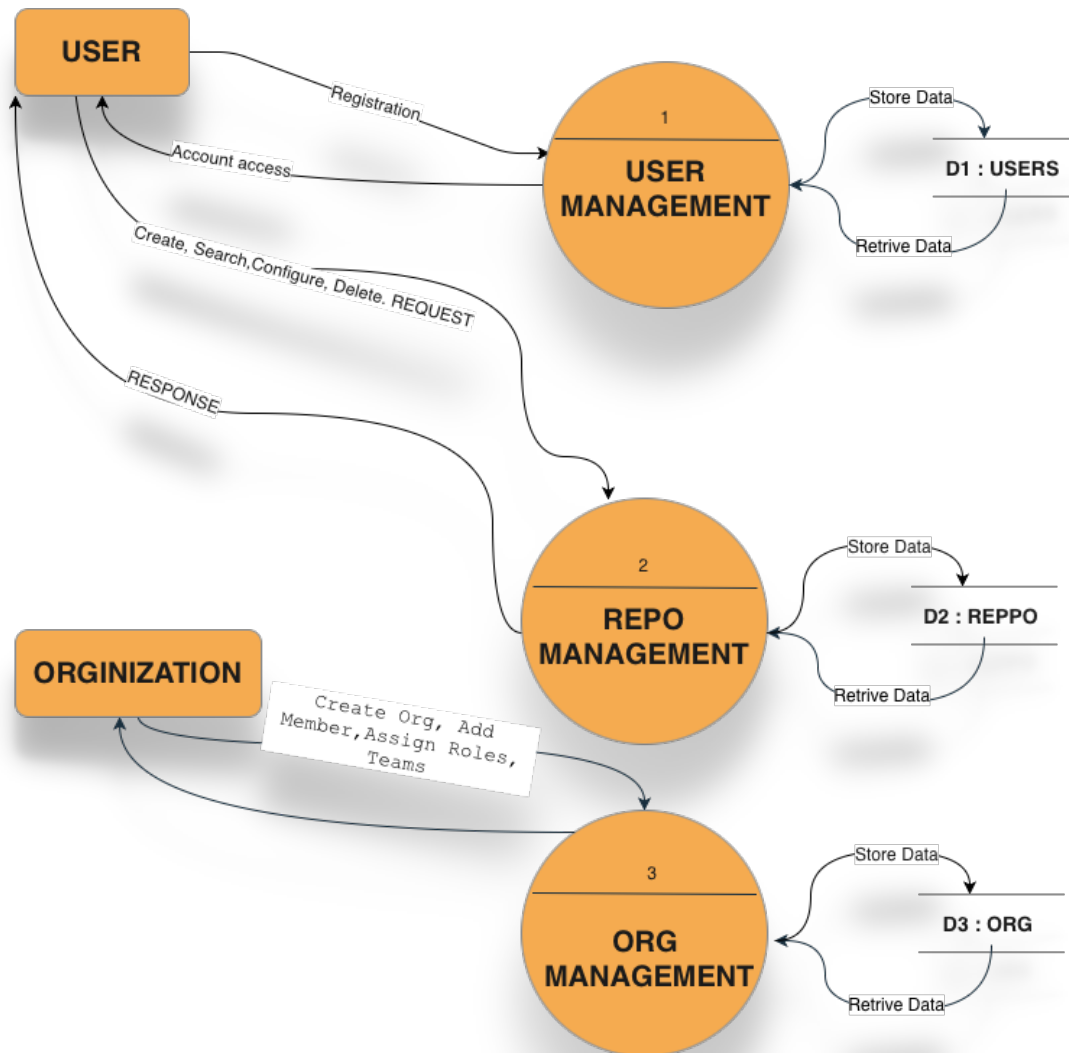
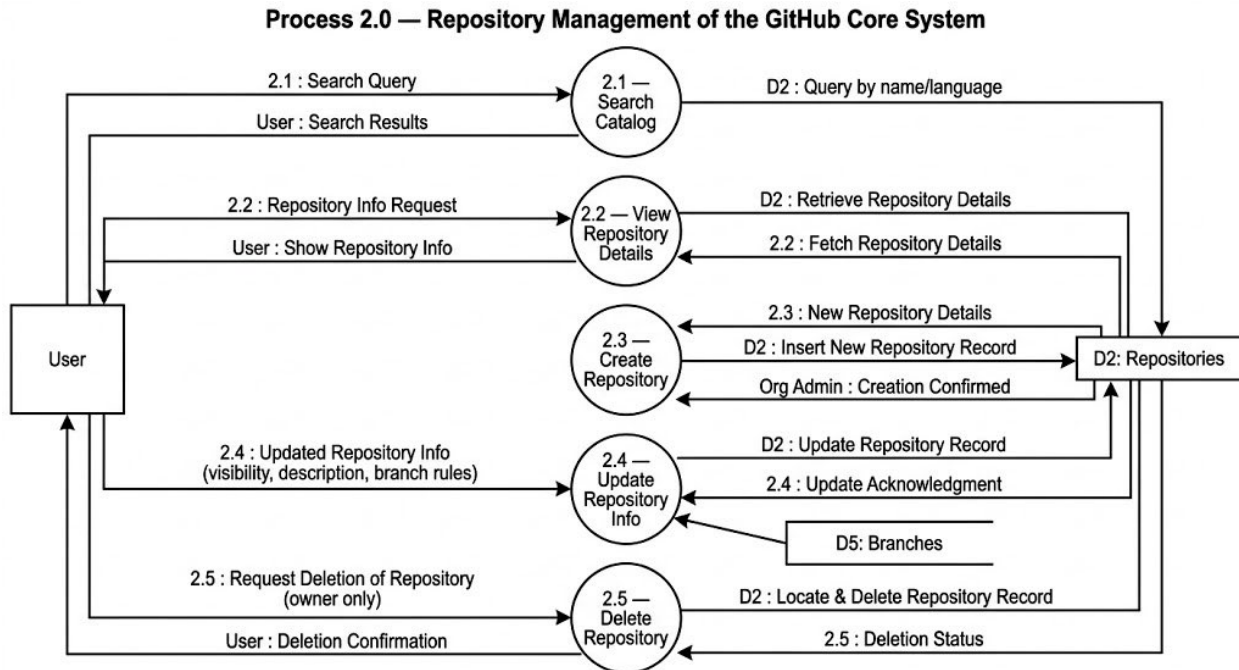


Fig: Level-0 DFD for GitHub Core System

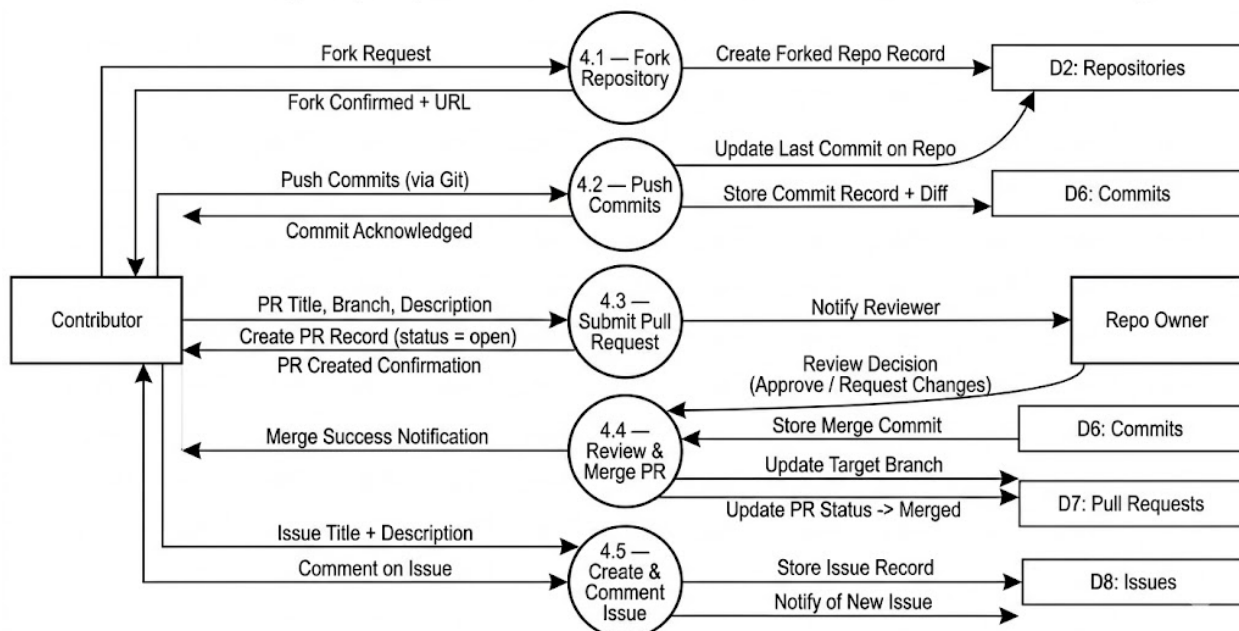
Level-1 DFD — Repository Management (Process 2.0)

This diagram expands Process 2.0 to show how repository data is created, searched, configured, and removed.



Level-1 DFD — Contribution Workflow (Process 4.0)

Level-1 Data Flow Diagram (DFD) for Process 4.0 — Contribution Workflow of the GitHub Core System



This diagram expands Process 4.0, detailing how contributors fork repositories, commit changes, submit pull requests, and interact with issues.

Level-2 DFD — Pull Request Processing (Sub-process 4.3-4.4)

This Level-2 DFD provides granular detail on how pull requests are validated, created, reviewed, and merged within the system.

Level-2 Data Flow Diagram (DFD) for Sub-process 4.3–4.4 — Pull Request Processing of the GitHub Core System

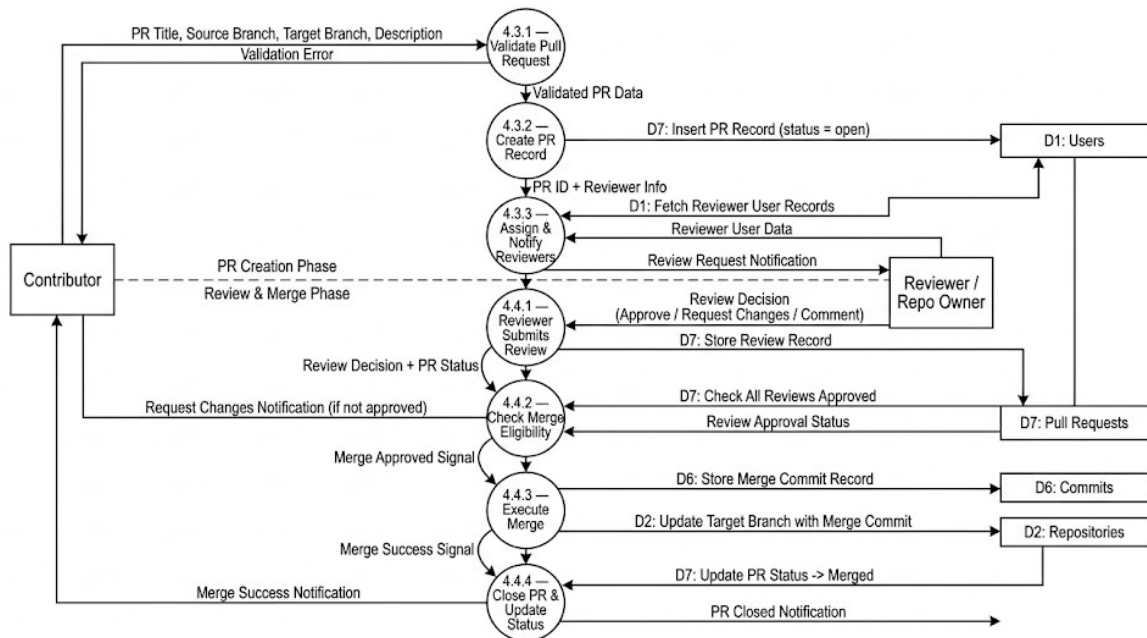


Fig: Level-2 DFD — Pull Request Processing

3.5 Entity Relationship Diagram (ERD)

The ERD models the complete database structure for the GitHub core system, showing the four primary entities and their relationships. Six tables are defined: users, organizations, repositories, contributors (junction), org_memberships (junction), pull_requests, commits, and issues.

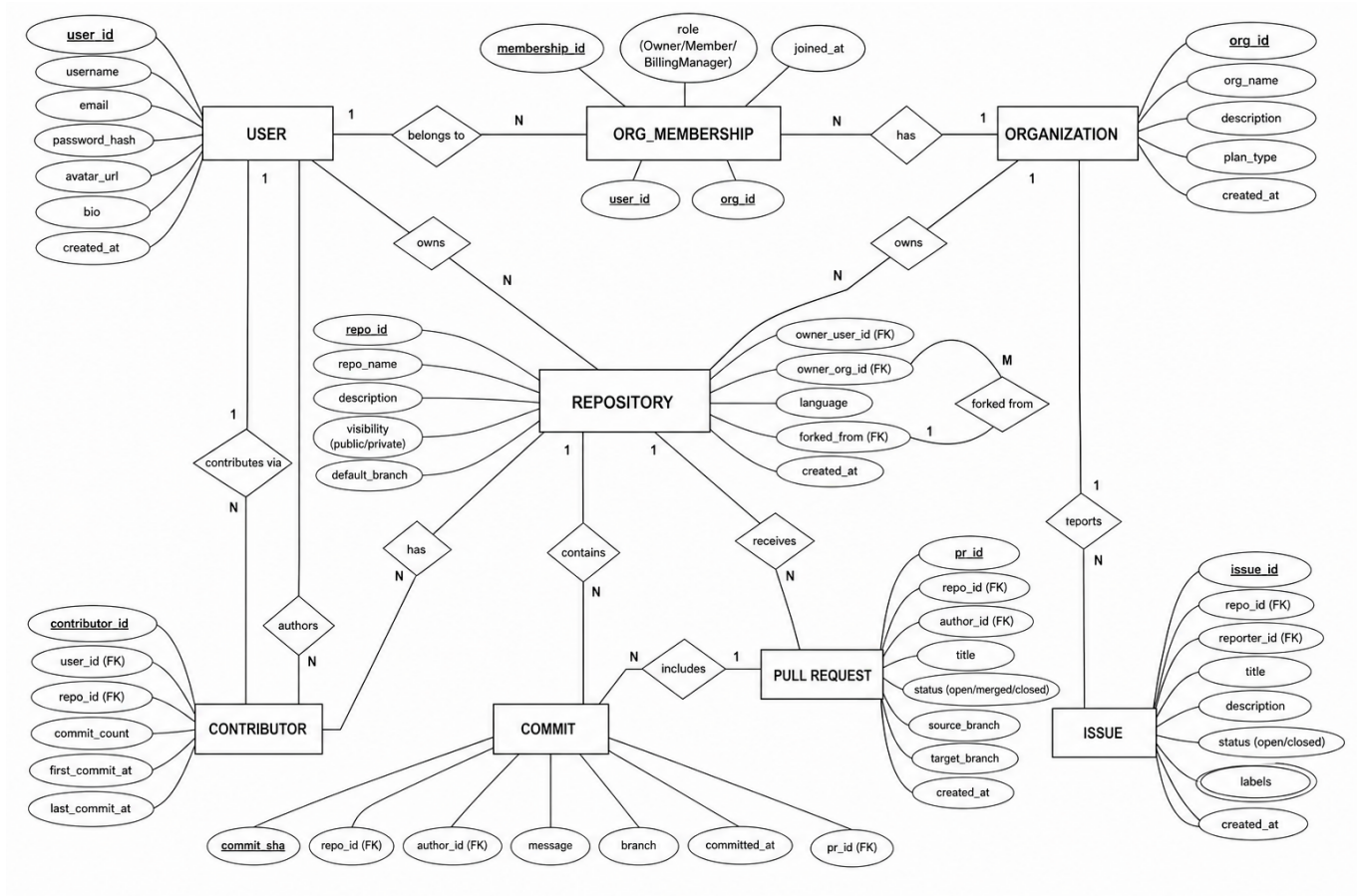
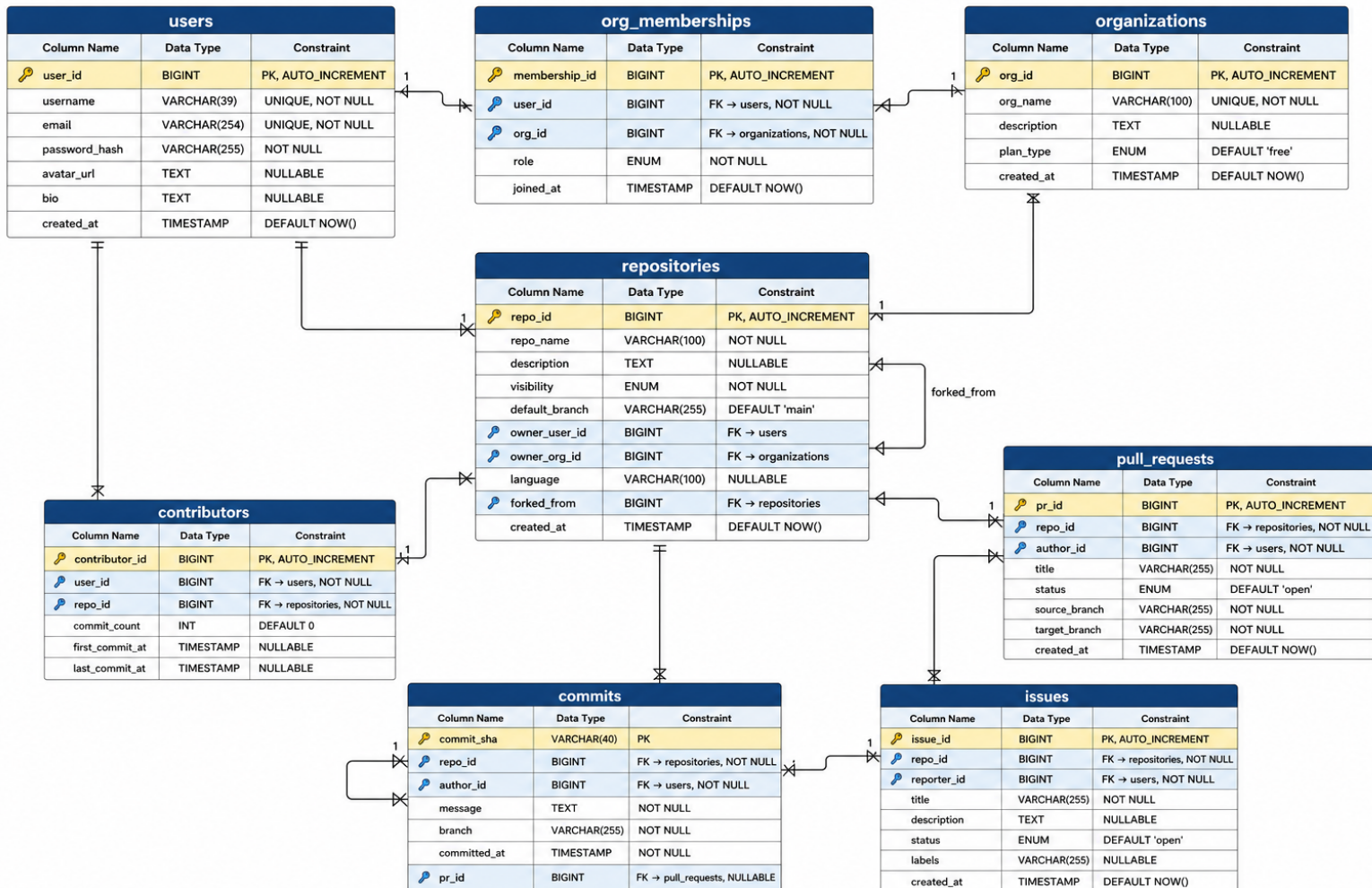


Fig: Entity Relationship Diagram for GitHub Core System

3.5.1 Physical Database Table Design

The following tables present the physical database schema derived from the ERD above.



3.6 Sequence Diagram

The Sequence Diagram below illustrates the Fork-and-Contribute workflow: a contributor forks a repository, pushes commits, submits a pull request, and the repository owner reviews and merges it.

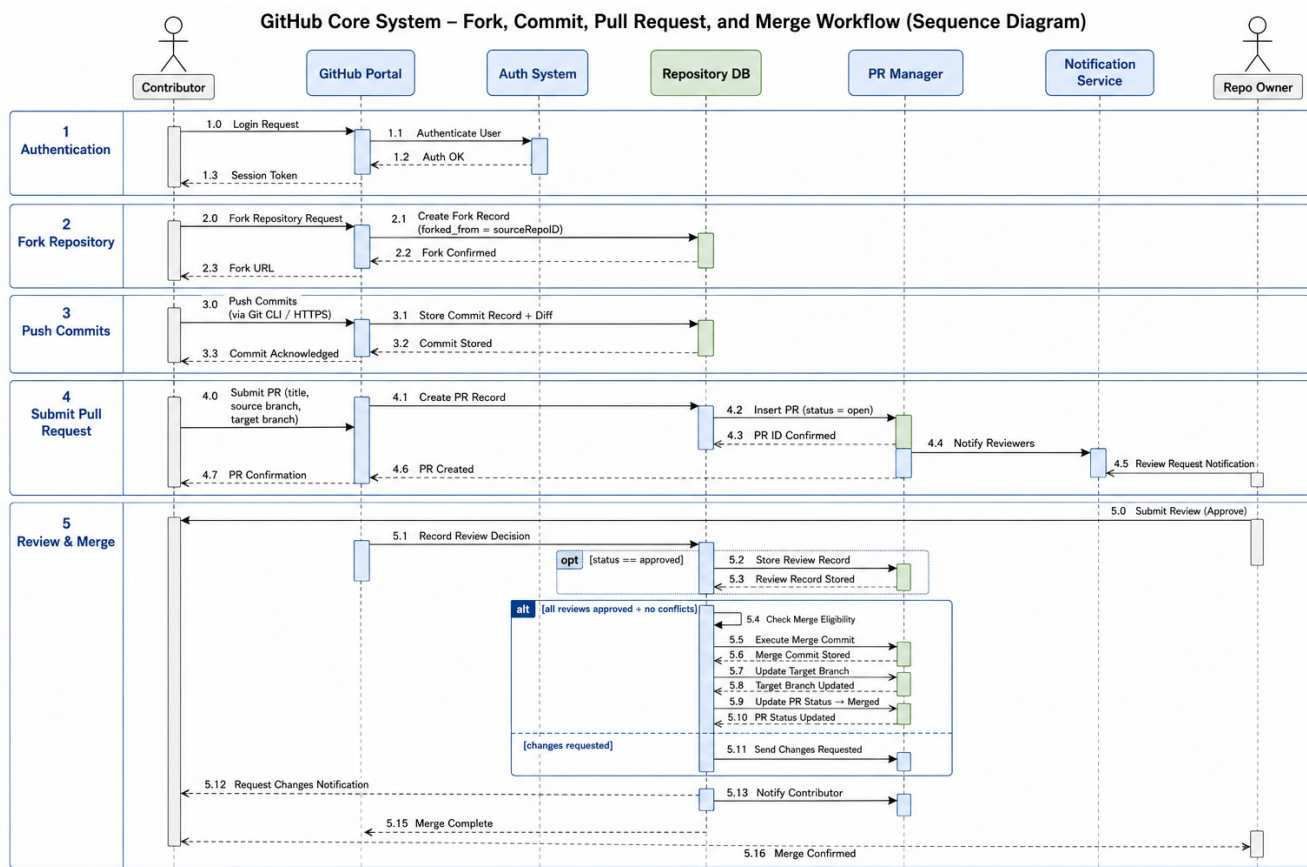


Fig: Sequence Diagram — Fork, Commit, Pull Request, and Merge Workflow

Chapter 4: Implementation and Maintenance

Implementation involves coding, testing, and integrating the system components based on design specifications. This includes database setup, API endpoint development, authentication service integration, and notification dispatch. Maintenance begins immediately after deployment to ensure ongoing stability and performance.

4.1 Pseudo Code

```

FUNCTION createRepository(userID, repoName, visibility, description)
{
  IF userExists(userID) THEN
    newRepo = { repoName, ownerUserID: userID, visibility,
                description, createdAt: NOW() }
    INSERT newRepo INTO repositories
    RETURN newRepo.repoID
  ELSE
    RETURN ERROR 'User not found'
}

```

```

FUNCTION forkRepository(contributorUserID, sourceRepoID)
{
  IF repositoryExists(sourceRepoID) AND
  isAccessible(contributorUserID, sourceRepoID) THEN
    forkedRepo = createRepository(contributorUserID,
    getRepoName(sourceRepoID), 'public', '')
    UPDATE forkedRepo SET forkedFrom = sourceRepoID
    addContributor(contributorUserID, forkedRepo.repoID)
    RETURN forkedRepo
  ELSE
    RETURN ERROR 'Source repository not found or inaccessible'
}

```

```

FUNCTION submitPullRequest(authorUserID, sourceRepoID,
                           targetRepoID, sourceBranch,
                           targetBranch, title)
{
  IF isContributor(authorUserID, sourceRepoID) THEN
    pr = { authorUserID, repoID: targetRepoID,
           sourceBranch, targetBranch, title,
           status: 'open', createdAt: NOW() }
    INSERT pr INTO pull_requests
    notifyReviewers(targetRepoID, pr.prID)
    RETURN pr.prID
  ELSE
    RETURN ERROR 'User is not a contributor to source repository'
}

```

```

FUNCTION mergePullRequest(reviewersUserID, prID)
{
  pr = getPullRequest(prID)

```

```

IF hasPermission(reviewerUserID, pr.repoID, 'write') THEN
  IF pr.status == 'open' THEN
    mergeCommit = executeMerge(pr.sourceBranch, pr.targetBranch)
    UPDATE pull_requests SET status = 'merged' WHERE prID = prID
    notifyAuthor(pr.authorUserID, 'PR merged successfully')
    RETURN mergeCommit
  ELSE
    RETURN ERROR 'PR is not in open state'
  ELSE
    RETURN ERROR 'Insufficient permissions to merge'
}

FUNCTION addOrgMember(adminUserID, orgID, newUserID, role)
{
  IF hasPermission(adminUserID, orgID, 'owner') THEN
    membership = { userID: newUserID, orgID,
                  role, joinedAt: NOW() }
    INSERT membership INTO org_memberships
    notifyUser(newUserID, 'Added to organization')
    RETURN SUCCESS
  ELSE
    RETURN ERROR 'Only org owners can add members'
}

```

4.2 Test Cases

The following test cases validate the core operations of the GitHub system across all four entities.

Test ID	Description	Test Data	Expected Result	Result
USR001	User registers with valid credentials	Username: dev_alice, Email: alice@mail.com	Account created; welcome notification sent; user redirected to dashboard	Pass / Fail
USR002	Registration with duplicate username	Username: dev_alice (already exists)	System returns 'Username already taken' error; no new record created	Pass / Fail
REPO001	User creates a public repository	RepoName: my-project, Visibility: Public	Repo created; appears in user profile; default branch initialized	Pass / Fail
REPO002	Non-owner attempts to delete repository	RepoID: 101, UserID: 55 (not owner)	System denies request with 403 Forbidden error	Pass / Fail

Test ID	Description	Test Data	Expected Result	Result
ORG001	Admin adds a new member to organization	OrgID: 10, NewUserID: 77, Role: Member	User added; membership record created; notification sent to new member	Pass / Fail
ORG002	Non-owner tries to add org member	OrgID: 10, UserID: 55 (member role)	System denies with 'Insufficient permissions' error	Pass / Fail
CONT001	Contributor forks a public repository	SourceRepoID: 201, ContributorID: 88	Forked repo created; forked_from set to 201; contributor record added	Pass / Fail
CONT002	Contributor submits a pull request	RepoID: 201, Branch: feature/login, Title: 'Add login page'	PR created with status 'open'; repo owner receives review notification	Pass / Fail
CONT003	Owner merges an approved pull request	PRID: 305, ReviewerID: 10 (write access)	Merge commit created; PR status updated to 'merged'; contributor notified	Pass / Fail
SYS001	System handles 500 concurrent repo requests	500 users simultaneously loading repos	All requests complete within 3 seconds; no crashes or stalls	Pass / Fail

Chapter 5: Conclusion

5.1 Conclusion

This report has presented a comprehensive system analysis of the GitHub core platform focused on four primary entities: Users, Organizations, Repositories, and Contributors. Through a three-level DFD hierarchy, a normalized ERD with physical table designs, a sequence diagram of the fork-to-merge workflow, and pseudo-code for key operations, the report demonstrates how GitHub's collaborative model can be systematically designed.

The system effectively addresses the core challenges of collaborative software development: distributed version-controlled code storage, role-based access management, transparent contribution workflows, and scalable organization structures. The pull request lifecycle and forking model, captured in the Level-2 DFD and sequence diagram, represent the heart of GitHub's value proposition as a collaborative development platform.

5.2 Lessons Learned

- Clear entity boundaries are critical: Separating User, Organization, Repository, and Contributor prevents architectural ambiguity and enables clean access control modeling.
- DFDs are most powerful when layered: Context diagrams provide breadth, Level-0 provides overview, and Level-1/2 expose the implementation-level logic.
- Junction tables enable M:N flexibility: The Contributor and OrgMembership tables show how many-to-many relationships carry meaningful attributes beyond simple linking.
- Permission modeling needs early design: Role-based access control at organization and repository levels must be designed upfront to avoid costly refactoring.
- Notification systems are cross-cutting: They touch every major workflow and should be treated as a dedicated service rather than embedded per-process logic.

5.3 Future Recommendations

- CI/CD Integration: Extend the system to include GitHub Actions, linking workflow runs to commits and pull requests for automated testing and deployment pipelines.
- AI-Powered Code Review: Integrate AI assistance for automated code suggestions and vulnerability detection within the pull request workflow.
- Advanced Analytics: Implement contribution dashboards for organizations to track developer activity, repository health metrics, and open-source engagement trends.
- Mobile Application: Develop a native mobile app for on-the-go repository browsing, issue management, and pull request review.
- Multi-Language Support: Extend the portal interface to multiple languages to improve accessibility for the global developer community.
- Dependency Graph Modeling: Map package dependencies between repositories to support vulnerability scanning and automated impact analysis.

References

1. E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. German, and D. Damian, "The Promises and Perils of Mining GitHub," in Proc. 11th Working Conf. on Mining Software Repositories (MSR), 2014, pp. 92-101.
2. J. Tsay, L. Dabbish, and J. Herbsleb, "Influence of Social and Technical Factors for Evaluating Contribution in GitHub," in Proc. 36th Int. Conf. on Software Engineering (ICSE), 2014, pp. 356-366.
3. C. Bird, P. Devanbu, and E. Keinan, "Does Distributed Development Affect Software Quality? An Empirical Case Study of Windows Vista," in Proc. 31st Int. Conf. on Software Engineering (ICSE), 2009, pp. 518-528.
4. GitHub Inc., "GitHub Documentation - About Organizations," GitHub Docs, 2024. [Online]. Available: <https://docs.github.com/en/organizations>.
5. GitHub Inc., "GitHub Documentation - About Repositories," GitHub Docs, 2024. [Online]. Available: <https://docs.github.com/en/repositories>.
6. L. Dabbish, C. Stuart, J. Tsay, and J. Herbsleb, "Social Coding in GitHub: Transparency and Collaboration in an Open Software Repository," in Proc. ACM Conf. on Computer-Supported Cooperative Work (CSCW), 2012, pp. 1277-1286.